

GUÍA TÉCNICA DE DESPLIEGUE EN ENTORNO REAL CLOUD (PRODUCCIÓN)

PROYECTO: SISTEMA WEB PWA DE GESTIÓN Y TOMA DE PEDIDOS - LEOFIT SOLUTIONS

UTP - CURSO INTEGRADOR II (100000S12F) | GRUPO 01

1. ARQUITECTURA DE DESPLIEGUE CLOUD MULTI-NIVEL

El sistema LeoFit está diseñado con una arquitectura modular desacoplada basada en microservicios y Clean Architecture, permitiendo un despliegue cloud de alto rendimiento, bajo costo y alta disponibilidad:

Componente	Tecnología	Proveedor Cloud Recomendado	Estrategia de Despliegue	URL de Producción
Frontend PWA	React 19 + Vite + Tailwind	**GitHub Pages** / **Vercel**	CI/CD Automático (GitHub Actions / Vercel Bot)	`https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/`
Backend API REST	Node.js 20 + TypeScript + Express	**Render.com** / **Railway**	Contenedor Docker Multi-Stage (`backend/Dockerfile`)	`https://leofit-backend-api.onrender.com`
Base de Datos	PostgreSQL 16 + Triggers	**Supabase** / **Neon Tech** / Render PG	Conexión SSL pooled (PgBouncer)	`postgresql://postgres:[PASSWORD]@[HOST]:6543/postgres`
Monitoreo & Logs	Healthcheck + Morgan	**Render Telemetry** / Sentry	Endpoint `/api/health` con probe HTTP cada 30s	`https://leofit-backend-api.onrender.com/api/health`

2. GUÍA PASO A PASO: DESPLIEGUE EN ENTORNO REAL

Opción 1: Despliegue Automatizado con GitHub Pages (Frontend) y Render (Backend)

Paso 1: Despliegue del Frontend en GitHub Pages

1. El repositorio ya incluye la GitHub Action oficial en ``.github/workflows/deploy.yml``.
2. En el repositorio de GitHub:
 - Ir a Settings > Pages.
 - En Build and deployment > Source, seleccionar Deploy from a branch.
 - Elegir la rama ``gh-pages`` y carpeta ``/`` (root).
 - Guardar. Cada ``git push main`` actualizará automáticamente el portal en producción en menos de 2 minutos.

Paso 2: Despliegue de la Base de Datos PostgreSQL en Supabase / Neon

1. Iniciar sesión en [Supabase](https://supabase.com) o [Neon](https://neon.tech).
2. Crear un nuevo proyecto denominado ``leofit-db``.
3. En el SQL Editor, pegar y ejecutar secuencialmente:
 - ``database/schema.sql`` (creación de tablas, constraints, triggers e índices).
 - ``database/seeds.sql`` (población de usuarios, roles, catálogo y pedidos).
4. Copiar la URI de conexión de producción con connection pooling transaccional habilitado (``Transaction Pooler, puerto 6543``).

Paso 3: Despliegue del Backend API REST en Render.com

1. Iniciar sesión en [Render.com](https://render.com).
2. Seleccionar New + > Blueprint (o Web Service).
3. Conectar el repositorio de GitHub: ``Leofit-Solutions-Grupo01/leofit-pedidos-sistema``.
4. El archivo ``render.yaml`` preconfigura automáticamente el servicio web y la base de datos gestionada.
5. Configurar las variables de entorno de producción:
 - ``NODE_ENV=production``
 - ``PORT=4000``
 - ``DATABASE_URL=postgresql://postgres:[PASSWORD]@[HOST]:6543/postgres?sslmode=require``
 - ``JWT_SECRET=[CLAVE_SEGURA_ALEATORIA_64_CHARS]``
 - ``CORS_ORIGIN=https://<dominio>`` (Nota: Fail-fast preventivo; si se despliega en producción sin esta variable o se usa ``*``, el servidor crashea por seguridad).
6. Presionar Create Web Service. Render compilará el código TypeScript y levantará el contenedor en segundos.

Opción 2: Despliegue Rápido de Frontend en Vercel

1. Ingresar a [Vercel](https://vercel.com) e importar el repositorio.
2. Si el root directory es el principal, el archivo ``vercel.json`` en la raíz enrutará automáticamente el build.

3. Si se define el root directory como `frontend/`:

- Framework Preset: Vite
- Build Command: `npm run build`
- Output Directory: `dist`

4. Presionar Deploy. Vercel aprovisionará un dominio HTTPS con CDN global Edge Network.

3. VERIFICACIÓN Y PRUEBAS EN PRODUCCIÓN (SMOKE TESTS)

Una vez completado el despliegue, validar la operatividad mediante los siguientes comandos:

```
# 1. Verificar estado del Backend y conexión con PostgreSQL
curl -i https://leofit-backend-api.onrender.com/api/health

# Respuesta esperada:
# HTTP/2 200 OK
# {"status":"UP","timestamp":"2026-09-25T...", "database":{"status":"connected"}}

# 2. Verificar catálogo público de productos
curl -i https://leofit-backend-api.onrender.com/api/products

# 3. Verificar servicio de interoperabilidad externa (APF3)
curl -X POST https://leofit-backend-api.onrender.com/api/external/whatsapp/notify \
-H "Content-Type: application/json" \
-d '{"phone":"51987654321", "orderCode":"LFT-001", "customerName":"Cliente Demo", "total":150.0, "status":"Recibido"}'
```

4. MATRIZ DE CUMPLIMIENTO DE CRITERIOS CLOUD EN LAS RÚBRICAS

Criterio Evaluado	Evidencia en el Proyecto	Estado
APF2 Criterio 4: Validación y Verificación en Cloud	Endpoints activos, conexión transaccional con PostgreSQL en nube, SSL TLS 1.3	**CUMPLE (2.0 / 2.0 pts)**
APF3 Criterio 4: Validación y Verificación Cloud v2	Pruebas funcionales e integración validadas contra endpoints en la nube	**CUMPLE (2.0 / 2.0 pts)**
APF3 Criterio 5: Sustentación con Demo Cloud	Frontend en GitHub Pages / Vercel accesible en vivo desde móviles y laptops de jurados	**CUMPLE (2.0 / 2.0 pts)**